

My Thought Process and Declaration

A concise account of the Machine Monitor MVP, its assumptions, and what I learned

I translated the assessment into a simple picture: three sensors report what a remote machine is doing, but the messenger carrying those reports is unreliable. A message may be repeated, delayed, delivered out of order, or never delivered. From that picture, I reduced the MVP to four decisions.

01 / VALIDATE Accept only a usable message ID, timestamp, recognized sensor, and finite numeric value.	02 / DEDUPLICATE Remember accepted IDs and ignore a message whose ID has already been seen.
03 / RESTORE TIME ORDER Buffer readings, order them by measurement time, and finalize them using watermark = highest timestamp - allowed lateness.	04 / CLASSIFY FRESH DATA Discard stale knowledge, then use power, temperature, and vibration to return IDLE, STARTING, RUNNING, WARNING, or UNKNOWN.

Read a line, ignore comments, validate four fields, create a Reading, reject duplicates and messages that are too late, sort accepted messages in the waiting buffer, move the watermark, release safe timestamps, update the latest sensor values, discard stale knowledge, classify the machine, and print one result per timestamp.

Why I shaped the MVP this way

The watermark initially sounded like a phrase reserved for distributed-systems engineers standing in front of three monitors. In practice it is a line in the sand: I have waited long enough for older messages, so I will finalize everything before this point. The five-second lateness window and thirty-second freshness window are documented MVP assumptions, not universal truths. I kept one file for input/output and one for the data model and monitoring logic. Restart snapshots, timers, multiple machines, and production-scale storage remain documented extensions. When required information is not fresh, the program returns UNKNOWN rather than guessing.

Declaration

I did not write the automated tests myself; I used an LLM to generate all of them. I ran them to check parsing, duplicates, out-of-order delivery, late messages, stale readings, warning priority, normal states, and one result per timestamp. They provided a useful safety net, but I do not want to present generated tests as proof that I independently mastered Rust testing in four days.

What I learned

The Rust Book was extremely useful for ownership, borrowing, enums, Result, collections, and error handling. Even after completing the assessment, I have the strange feeling that I did a lot while still knowing very little. In roughly four days with Rust, I repeatedly entered gray areas where I understood what the program required without yet understanding every language-level reason behind it. The borrow checker often felt like an unusually strict reviewer, and `.clone()` occasionally looked like a peace treaty until I realized I was paying one credit card with another.

What allowed me to finish was previous exposure to small projects in C, C++, Python, JavaScript, Bash, and Lua. Parsing input, trimming noise, rejecting outliers, preserving state, and protecting core logic from unreliable input were familiar ideas wearing different syntax and stricter rules. This assessment made me realize how connected software concepts are, while also showing me how much distance remains between recognizing an idea and truly mastering it. I am submitting a working MVP and an honest account of what I understood, what I assumed, where I used assistance, and what I still need to learn.